

Name:- Ashutosh Kumar singh  
2404921540056 , A1 , Cse (Ds)

## Module - 2

### Q17 Fraud Transaction Detection

Linear search check each element one by one from beginning until the required element is found or the array ends.

Logic Loop visits every element  
array[i] == search check whether the current element matches the target.  
If matched, set flag & stop searching.

```
public class main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int arr[] = new int[n];
        for(int i = 0; i < n; i++)
        {
            arr[i] = sc.nextInt();
        }
        int search = sc.nextInt();
        int flag = 0;
        for(int i = 0; i < n; i++)
        {
            if(arr[i] == search)
            {
                flag = 1;
                break;
            }
        }
        if(flag == 1)
            System.out.println("Fraud Transaction Found");
        else
            System.out.println("Transaction Not Found");
    }
}
```

```
import java.util.Scanner;

public class FraudTransactionDetection {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int key = sc.nextInt();
        boolean found = false;

        for(int i = 0; i < n; i++) {
            if(arr[i] == key) {
                found = true;
                break;
            }
        }

        if(found)
            System.out.println("Fraud Transaction Found");
        else
            System.out.println("Transaction Not Found");
    }
}
```

"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\5  
1201 1305 1450 1408 1502  
1450  
Fraud Transaction Found  
Process finished with exit code 0

Server log search

2.1 Binary search works only on a sorted array. It repeatedly divides the array into two halves until the element is found.

Logic

- (i) Set low = 0 & high = n-1.
- (ii) Find middle element.
- (iii) If middle equals target → found.
- (iv) If target is greater → search right half.
- (v) Else search left half.
- (vi) Repeat until found or array ends.

```

public class main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int arr[] = new int[n];
        for(int i=0; i<n; i++)
        {
            arr[i] = sc.nextInt();
        }
        int search = sc.nextInt();
        int low = 0;
        int high = n-1;
        int flag = 0;
        while(low <= high)
        {
            int mid = (low + high) / 2;
            if(arr[mid] == search)
            {
                flag = 1;
                break;
            }
        }
    }
}

```

```

else if(arr[mid] < search)
    low = mid + 1;
else
    high = mid - 1;
}
if(flag == 1)
    System.out.println("Log Found");
else
    System.out.println("Log not found");
}

```

```

public class ServerLogSearch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int key = sc.nextInt();

        int low = 0, high = n - 1;
        boolean found = false;

        while(low <= high) {
            int mid = (low + high) / 2;

            if(arr[mid] == key) {
                found = true;
                break;
            }
            else if(arr[mid] < key) {
                low = mid + 1;
            }
            else {
                high = mid - 1;
            }
        }

        if(found)
            System.out.println("Log Found");
        else
            System.out.println("Log Not Found");
    }
}

```

Output:

```

7
1001 1005 1010 1015 1020 1030 1040
1015
Log Found
Process finished with exit code 0

```

Q3 Sort Employee Salaries

→ Bubble sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.

→ Largest element moves to the end in each pass like a bubble rising in water.

Logic

- (i) Compare adjacent elements
- (ii) Swap if left element is greater
- (iii) Repeat for all elements
- (iv) Continue until array is sorted.

```

public class main
{
    public static void main (String[] args)
    {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int arr[] = new int[n];

        for (int i = 0; i < n; i++)
        {
            arr[i] = sc.nextInt();
        }

        for (int i = 0; i < n - 1; i++)
        {
            for (int j = 0; j < n - 1 - i; j++)
            {
                if (arr[j] > arr[j + 1])
                {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }

        for (int i = 0; i < n; i++)
        {
            System.out.print(arr[i] + " ");
        }
    }
}

```

```

package Workshop2;

import java.util.Scanner;

public class SortEmployeeSalaries {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }

        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}

```

Output:

```

5
45000 32000 55000 28000 60000
28000 32000 45000 55000 60000
Process finished with exit code 0

```



04

Sort website response time

Insertion sort inserts each element into its correct in the already sorted part of the array.

- ① Assume first element is sorted
- ② Take next element
- ③ Compare with previous elements
- ④ Insert it at correct position
- ⑤ Repeat for all elements.

```

public class main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++)
        {
            arr[i] = sc.nextInt();
        }

        for (int i = 1; i < n; i++)
        {
            int key = arr[i];
            int j = i - 1;

            while (j >= 0 && arr[j] > key)
            {
                arr[j+1] = arr[j];
                j--;
            }
            arr[j+1] = key;
        }

        for (int i = 0; i < n; i++)
        {
            System.out.print(arr[i] + " ");
        }
    }
}

```

```

package Workshop2;
import java.util.Scanner;
public class SortWebsiteResponseTime {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        for(int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;

            while(j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }

            arr[j + 1] = key;
        }

        for(int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}

```

```

"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\
5
250 120 320 180 100
100 120 180 250 320
Process finished with exit code 0

```

## Q5 Prioritize Bug Reports

Selection sort repeatedly finds the smallest element from the unsorted part and places it at the beginning.

### Logic

- (i) Find smallest element
- (ii) Swap it with first position
- (iii) Repeat for remaining element
- (iv) Continue until sorted.

```

public class main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int arr[] = new int[n];
        for (int i = 0; i < n; i++)
        {
            arr[i] = sc.nextInt();
        }
        for (int i = 0; i < n - 1; i++)
        {
            int min = i;
            for (int j = i + 1; j < n; j++)
            {
                if (arr[j] < arr[min])
                {
                    min = j;
                }
            }
            int temp = arr[i];
            arr[i] = arr[min];
            arr[min] = temp;
        }
        for (int i = 0; i < n; i++)
        {
            System.out.print(arr[i] + " ");
        }
    }
}
    
```

```

package Workshop2;
import java.util.Scanner;
public class PrioritizeBugReport {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        for(int i = 0; i < n - 1; i++) {
            int minIndex = i;

            for(int j = i + 1; j < n; j++) {
                if(arr[j] < arr[minIndex]) {
                    minIndex = j;
                }
            }

            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }

        for(int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
    
```

"C:\Program Files\Java\jdk-25.0.2\bin\jav  
5  
9 2 8 1 5  
1 2 5 8 9  
Process finished with exit code 0